

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
CSA17 - ARTIFICIAL INTELLIGENCE

Experiment No. 10: Python Program to Implement the A* Algorithm

Aim

To write a Python program to implement the A* search algorithm for finding the shortest path between a start cell and a goal cell in a grid containing obstacles, using the Manhattan distance as the heuristic function.

Algorithm

1. Start.
2. Represent the environment as a 2-D grid, where 0 denotes an open (walkable) cell and 1 denotes an obstacle.
3. Read the start position and the goal position as (row, column) coordinates on the grid.
4. Define a heuristic function $\text{heuristic}(a, b)$ that computes the Manhattan distance $|a.\text{row} - b.\text{row}| + |a.\text{col} - b.\text{col}|$ between two cells.
5. Create a priority queue (min-heap) and insert the start node with $g = 0$ (cost from start) and $f = g + \text{heuristic}(\text{start}, \text{goal})$.
6. Create a visited set to keep track of expanded nodes.
7. While the priority queue is not empty: a) Pop the node with the lowest f value. b) If this node is the goal, return the path and its total cost. c) Otherwise, mark the node visited and generate its valid neighbors (Up, Down, Left, Right) that lie within the grid and are not obstacles.
8. For each unvisited neighbor, compute $g_{\text{new}} = g + 1$ and $f_{\text{new}} = g_{\text{new}} + \text{heuristic}(\text{neighbor}, \text{goal})$, then push it into the priority queue with the extended path.
9. If the priority queue is exhausted without reaching the goal, report that no path exists.
10. Print the path found, its total cost, and display the grid with the path marked.
11. Stop.

Pseudocode

```
BEGIN
  INPUT: grid, start, goal
  CREATE priority_queue PQ
  INSERT (f=heuristic(start,goal), g=0, start, path=[start]) into PQ
  CREATE VISITED = {}

  WHILE PQ is not empty DO
    f, g, current, path = PQ.pop_min_f()

    IF current == goal THEN
      PRINT "Path found", path, g
      BREAK
    END IF

    IF current IN VISITED THEN
      CONTINUE
```

```

END IF
ADD current TO VISITED

FOR each neighbor (Up, Down, Left, Right) of current DO
  IF neighbor is inside grid AND grid[neighbor] == 0
    AND neighbor NOT IN VISITED THEN
      g_new = g + 1
      f_new = g_new + heuristic(neighbor, goal)
      PQ.push(f_new, g_new, neighbor, path + [neighbor])
    END IF
  END FOR
END WHILE

IF goal not reached THEN
  PRINT "No path found"
END IF
END

```

Result

The Python program to implement the A* algorithm was executed successfully. For a 5x5 grid containing obstacles, with the start at (0,0) and the goal at (4,4), the program used the Manhattan distance heuristic to guide the search and found the shortest path of cost 8, avoiding all obstacle cells. The path and the grid with the route marked were displayed as output. Thus, the A* search algorithm was implemented and verified successfully using Python.

Output Screenshot:

```

Terminal - astar.py

$ python3 astar.py
Grid (0 = open, 1 = obstacle):
[0, 0, 0, 0, 0]
[0, 1, 1, 1, 0]
[0, 0, 0, 1, 0]
[1, 1, 0, 1, 0]
[0, 0, 0, 0, 0]

Start: (0, 0)   Goal: (4, 4)

Path found with cost 8!

Path: (0, 0) -> (0, 1) -> (0, 2) -> (0, 3) -> (0, 4) -> (1, 4) -> (2, 4) -> (3, 4)

Grid with path marked (* = path, # = obstacle):

* * * * *
. # # # *
. . . # *
# # . # *
. . . . *

A* algorithm executed successfully!

```